

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓名：张序鸿

学院：竺可桢学院

专业：计算机科学与技术

学号：3230103265

指导教师：杨莹春

2024 年 12 月 15 日

浙江大学实验报告

课程名称: 计算机组成与设计 实验类型: 综合

实验项目名称: 流水线处理器 — 冒险与 stall

学生姓名: 张序鸿 专业: 混合班 学号: 3230103265

指导老师: 杨莹春、潘之杰 助教: 马致远

实验地点: 东 4-509 实验日期: 2024 年 12 月 11 日

一、实验目的和要求

- 1.理解流水线 CPU 的基本原理和组织结构
- 2.掌握五级流水线的工作过程和设计方法
- 3.理解流水线 CPU 停机的原理与解决办法
- 4.设计流水线测试程序

二、实验内容和原理

内容:

任务一: 集成设计利用 stall 解决冒险的流水线 CPU

- 设计冒险检测及 stall 消除冒险的流水线 CPU
- 替换 lab05-3 的 CPU 为本实验集成的带 stall 处理的流水线 CPU

任务二: 设计流水线 stall 测试方案并完成测试

原理：

2.1 流水线冒险基本概念

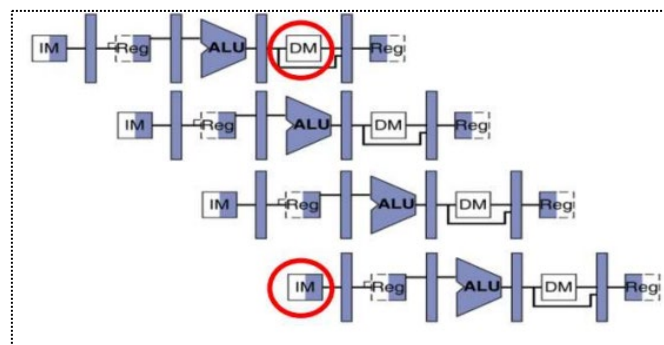
流水线冒险指在下一个时钟周期中下一条指令无法正常执行，引起冒险的原因有多种，主要分为以下三种：

- 结构冒险 (Structural hazard)：硬件不支持多条指令在同一时钟周期执行
- 数据冒险 (Data hazard)：当前指令的执行需要等待前一条指令的数据结果
- 控制冒险 (Control hazard)：指令非顺序执行而导致下一条执行的指令不是真实期望的。

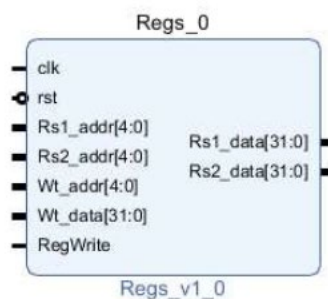
2.2 结构冒险 (Structural hazard)：

也称为硬件资源冲突，指流水线执行期间，两条及以上指令同一时间对同一个硬件资源发起使用的请求。

常见的结构冒险主要发生在 Memory conflicts(指令和数据)、Register conflicts。此次流水线中指令(PC 储存)、数据(Data Memory 储存)，前者(见下图)不会发生。



同时此次流水线中 Register 的读写端口分离，因此在 WB 和 ID 中对 regs 的同时读写并不会冲突。



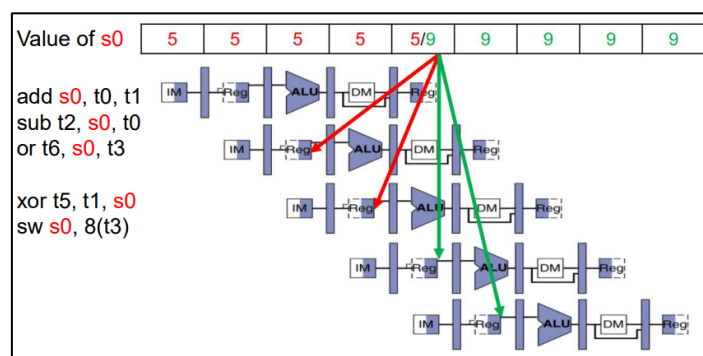
总而言之，本次中断实验不考虑结构冒险的存在。

2.3 数据冒险 (Data hazard):

指后一条指令的执行需要前一条指令的执行结果，而此时前一条指令的结果还未产生，主要有三种情况（指令 i 先于 j）：

- **RAW (Read After Write)**: 当指令 i 写回结果之前，指令 j 已经对此结果发起读操作；
- **WAW (Write After Write)**: 当指令 i 写回结果之前，指令 j 已经将结果写回；
- **WAR (Write After Read)**: 当指令 i 读取数据之前，指令 j 已经将结果写回。

本次实验主要处理 RAW 数据冒险，如下情况：



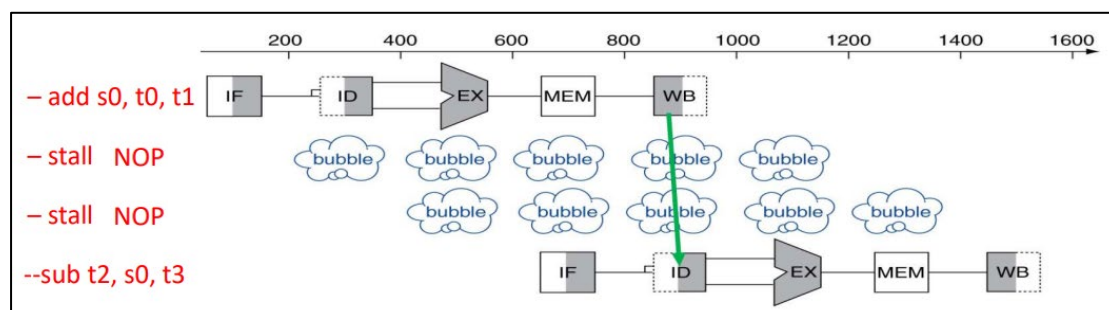
指令 SUB 和 OR 的源操作数都依赖于 ADD 的计算结果，若不采取应对措施，则在 S₀ 结果写回之前，错误的值会被提前读走（xor 时寄存器前写后读）。

2.3.1 数据冒险解决方法：

(1) Solution 1 : Stalling or NOP (• 本实验采用 •)

• 硬件解决方法：流水线阻塞 (stall)，使数据相关的后续指令延迟执行，也称为插入气泡 (bubble)。

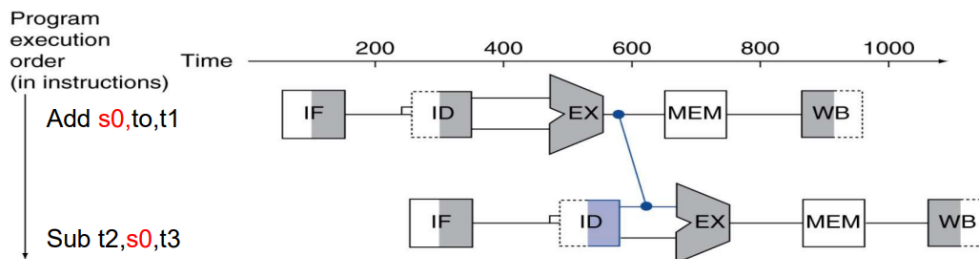
• 软件解决方法：插入空操作 (NOP: addi x0, x0, x0)，使数据相关的后续指令延迟执行。



两种方式虽然都使得流水线能够正常的执行操作，但延迟下一条指令执行的方式使 CPU 性能大打折扣。

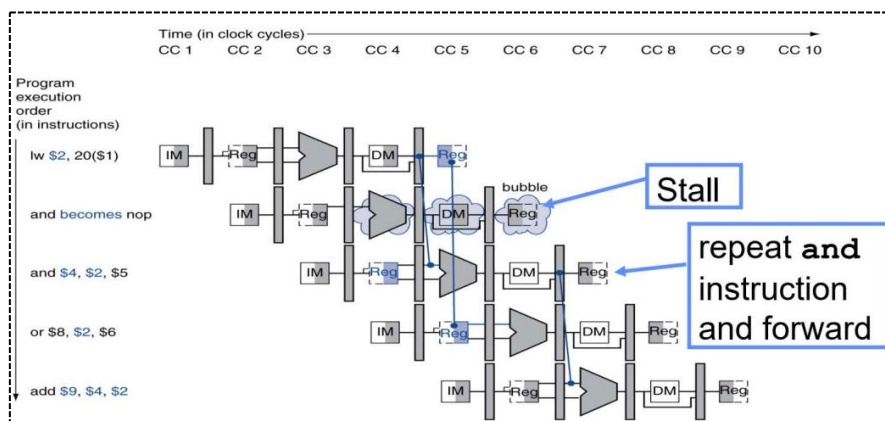
(2) Solution 2 : Forwarding

前递 (forwarding), 又称为旁路 (bypassing) 将数据通路生成的中间数据直接往前传递到 ALU 的输入端, 参与下一条指令的运算。



如上图, 将 ADD 指令执行阶段的输出值 (ALU 结果) 前递到 SUB 指令的执行阶段的输入, 替换 SUB 指令在第二阶段读出的寄存器 S0 的值 (无需等待寄存器的值被写回, 避免 RAW)。

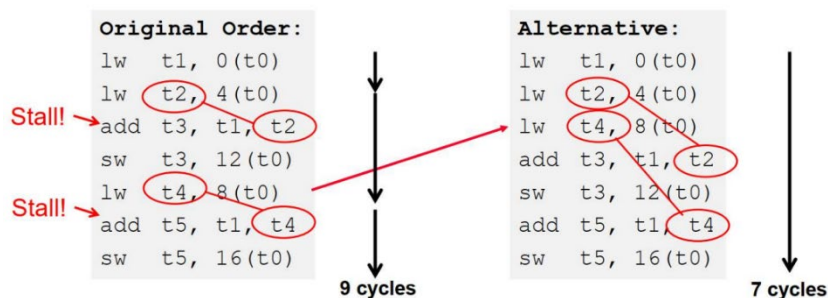
但一次前递只能解决数学运算之间的 Data Hazard, 当一条 load 指令后为一条需要使用其结果的 R 型指令后, 直接前递是无效的 (load 后数据在 Datamemory 中读取, 而非在 ALU 中, 则 Stall 无法避免的)。



(3) Solution 3 : Scheduling

编译器进行指令顺序调整来解决数据冒险。

• eg: RISC-V code for $A[3] = A[0] + A[1]$;
 $A[4] = A[0] + A[2]$



2.3.2 数据冒险检测机制:

(1) Mem Hazard:

(k) th 指令访存 (Ex) 阶段写回寄存器和 (k+1) th 指令译码 (ID) 阶段源寄存器相同, 同时 (k) th 指令数据写回寄存器 (EX/MEM.RegWrite=1)。

- if (EX/MEM.RegWrite and Rs1_used) and (ID.RegisterRs1 $\neq$ 0)
and (EX/MEM.RegisterRd = ID.RegisterRs1)) **Data_stall = 1**
- if (EX/MEM.RegWrite and Rs2_used) and (ID.RegisterRs2 $\neq$ 0)
and (EX/MEM.RegisterRd = ID.RegisterRs2)) **Data_stall = 1**

(2) Ex Hazard :

(k) th 指令执行 (Ex) 阶段写回寄存器和 (k+1) th 指令译码 (ID) 阶段源寄存器相同, 同时 (k) th 指令将有数据写回寄存器 (ID/EX.RegWrite=1)。

- if (ID/EX.RegWrite and Rs1_used) and (ID.RegisterRs1 $\neq$ 0)
and (ID/EX.RegisterRd = ID.RegisterRs1)) **Data_stall = 1**
- if (ID/EX.RegWrite and Rs2_used) and (ID.RegisterRs2 $\neq$ 0)
and (ID/EX.RegisterRd = ID.RegisterRs2)) **Data_stall = 1**

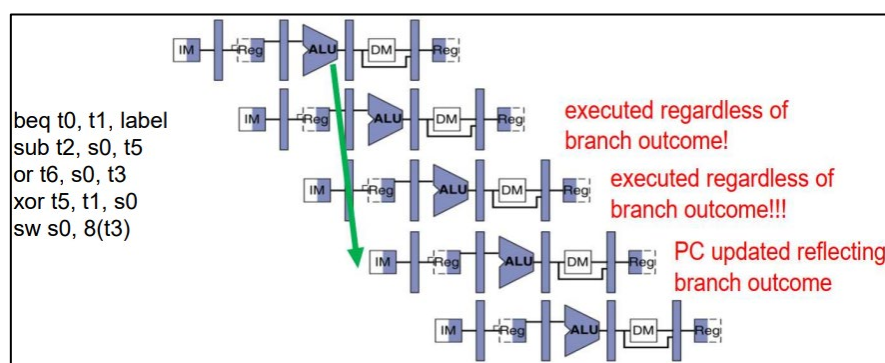
检测到 Data Hazard , 拉高 Data_stall 标志信号, 此时采取停机操作, 在译码和执行之间插入 NOP 指令:

```
if (Data_stall == 1'b1) begin
    en_IF = 0;
    en_IFID = 0;
    NOP_IDEX = 1;
end
else begin
    en_IF = 1;
    en_IFID = 1;
    NOP_IDEX = 0;
end
```

2.4 控制冒险

当 PC 值不按顺序执行时,流水线中指令的正常执行会被阻塞,常见的如 beq、bne、jal 等跳转指令。

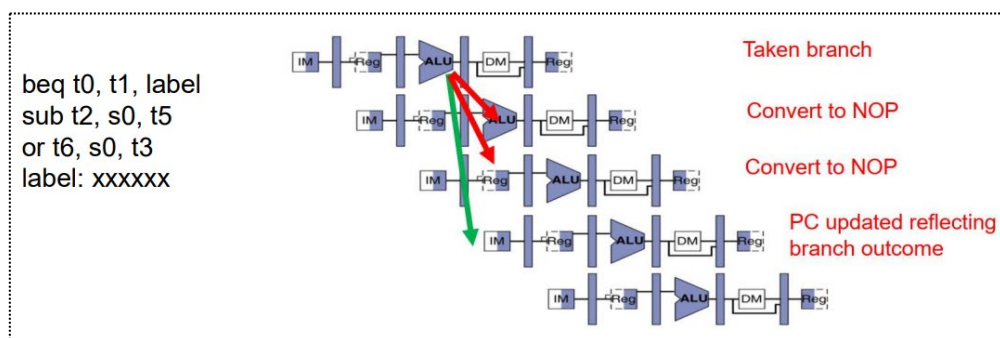
如图, beq 指令 (原计划在 ALU 计算结束后跳转到 xor 指令) 执行之后, 下一条指令应该等待其执行完成才知道目标地址, 此时若不采取措施, pc 会顺序的开始 sub 操作, 显然不是期望的结果:



2.4.1 控制冒险解决方法

(1) Solution 1 : Stalling or NOP (• 本实验采用 •)

采用类似 Data Hazard 的解决, 采用插入 NOP 的方式解决 (同样, 容易执行, 但是对 Pipeline 性能影响较大)。



(2) Solution 2 : Branch Prediction

分支预测 (Branch Prediction): 分为静态预测 (简单预测分支指令的条件总是满足或不满足) 和动态预测 (根据实际执行情况动态调整预测位), 预测准确则无时间损失, 若预测不准确则对分支后不该执行的指令进行冲刷。

从而最大限度减少控制冒险对性能的影响。

2.4.2 控制冒险检测机制

本实验会改变 PC 的指令为 beq;bne;jal; 因此只需检测这三条指令（采用拓展后的 SCPU_ctrl，因此 Jump 考虑了 jalr）的特征信号即可：

```
if ((Branch_ID || BranchN_ID || Jump_ID != 2'b00) ||  
    // 译码阶段  
    (Branch_out_IDEX || BranchN_out_IDEX || Jump_out_IDEX != 2'b00) ||  
    // 执行阶段  
    (Branch_out_EXMem || BranchN_out_EXMem || Jump_out_EXMem != 2'b00))  
    // 访存阶段  
begin  
    Control_stall = 1'b1;  
end  
else  
    Control_stall = 1'b0;
```

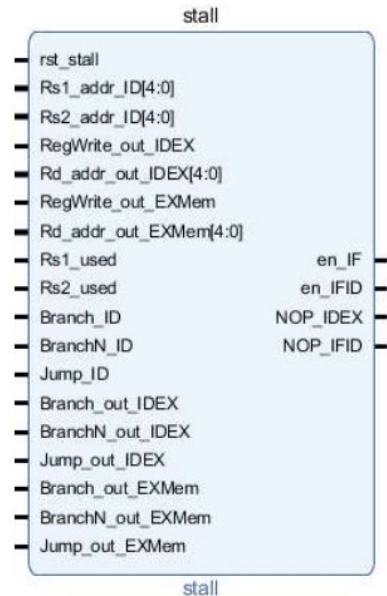
检测到 Control Hazard，拉高 Control_stall 标志信号，此时采取停机操作，在取指和译码之间插入 NOP 指令：

```
if (Control_stall == 1'b1)  
    NOP_IFID = 1;  
else  
    NOP_IFID = 0;
```


三、实验过程和数据记录

3.1 集成设计利用 stall 解决冒险的流水线 CPU:

(1) Stall —— 冒险检测及处理单元:



```
`timescale 1ns / 1ps
module stall(
    input rst_stall, //复位
    input RegWrite_out_IDEX, //执行阶段寄存器写控制
    input [4:0]Rd_addr_out_IDEX, //执行阶段寄存器写地址
    input RegWrite_out_EXMem, //访存阶段寄存器写控制
    input [4:0]Rd_addr_out_EXMem, //访存阶段寄存器写地址
    input [4:0]Rs1_addr_ID, //译码阶段寄存器读地址 1
    input [4:0]Rs2_addr_ID, //译码阶段寄存器读地址 2
    input Rs1_used, //Rs1 被使用
    input Rs2_used, //Rs2 被使用
    input Branch_ID, //译码阶段 beq
    input BranchN_ID, //译码阶段 bne
    input [1:0]Jump_ID, //译码阶段 jal
    input Branch_out_IDEX, //执行阶段 beq
    input BranchN_out_IDEX, //执行阶段 bne
    input [1:0]Jump_out_IDEX, //执行阶段 jal
    input Branch_out_EXMem, //访存阶段 beq
    input BranchN_out_EXMem, //访存阶段 bne
    input [1:0]Jump_out_EXMem,
    output reg en_IF, //流水线寄存器的使能及 NOP 信号
    output reg en_IFID,
    output reg NOP_IFID,
    output reg NOP_IDEX);
```

```

reg Data_stall, Control_stall;

always@(*) begin
    if(rst_stall) begin
        en_IF = 1;
        en_IFID = 1;
        NOP_IFID = 0;
        NOP_IDEX = 0;
    End

    if ((Rs1_used && (Rs1_addr_ID == Rd_addr_out_IDEX) &&
(Rs1_addr_ID != 5'h0) && RegWrite_out_IDEX) ||
        (Rs2_used && (Rs2_addr_ID == Rd_addr_out_IDEX) &&
(Rs2_addr_ID != 5'h0) && RegWrite_out_IDEX) ||
        (Rs1_used && (Rs1_addr_ID == Rd_addr_out_EXMem) &&
(Rs1_addr_ID != 5'h0) && RegWrite_out_EXMem) ||
        (Rs2_used && (Rs2_addr_ID == Rd_addr_out_EXMem) &&
(Rs2_addr_ID != 5'h0) && RegWrite_out_EXMem))
        begin
            Data_stall = 1'b1;
        end
    else
        Data_stall = 1'b0;
        if ((Branch_ID || BranchN_ID || Jump_ID != 2'b00) ||
            (Branch_out_IDEX || BranchN_out_IDEX || Jump_out_IDEX != 2'b00) ||
            (Branch_out_EXMem || BranchN_out_EXMem || Jump_out_EXMem != 2'b00))
            Control_stall = 1'b1;
        else
            Control_stall = 1'b0;
        if (Data_stall == 1'b1) begin
            en_IF = 0;
            en_IFID = 0;
            NOP_IDEX = 1;
        end
        else begin
            en_IF = 1;
            en_IFID = 1;
            NOP_IDEX = 0;
        end
        if (Control_stall == 1'b1)
            NOP_IFID = 1;
        else
            NOP_IFID = 0;
    end
end

```

根据 Stall 中所要求的内容，涉及了寄存器堆的读写地址信号，以及 Rs1、Rs2 的使用情况，因此译码阶段 Pipeline_ID 需要微调。

同时检测到 Hazard 后，我们得到了 en_IF、en_IFID、NOP_IDEX、NOP_IFID 等信号，因此需要对流水线各级寄存器（特别是 IF_reg_ID、ID_reg_EX 两个需要信号中断的寄存器）进行修改。

（2） IF_reg_ID：

• 接口修改：

```
module IF_reg_ID(  
    input clk_IFID,  
    input rst_IFID,  
    input en_IFID,  
    input NOP_IFID, // 插入 NOP 使能  
    input [31:0]PC_in_IFID,  
    input [31:0]inst_in_IFID,  
    output reg [31:0]PC_out_IFID,  
    output reg [31:0]inst_out_IFID,  
    output reg valid_IFID  
);
```

• 逻辑修改：

```
always @(posedge clk_IFID or posedge rst_IFID)  
    if (rst_IFID == 1'b1) begin  
        PC_out_IFID <= 32'h0;  
        inst_out_IFID <= 32'h0;  
        valid_IFID <= 1'b1;  
    end  
    else if(NOP_IFID) begin  
        PC_out_IFID <= 32'h0;  
        inst_out_IFID <= 32'h00000013; // 插入空指令 add x0, x0, x0  
        valid_IFID <= 1'b0;  
    end  
    else if (en_IFID == 1'b1) begin  
        PC_out_IFID <= PC_in_IFID;  
        inst_out_IFID <= inst_in_IFID;  
        valid_IFID <= 1'b1;  
    end  
end
```

(3) Pipeline_ID :

从 Stall 模块中知道，我们需要译码时增加以下四个信号：

```
input [4:0]Rs1_addr_ID, //译码阶段寄存器读地址 1
input [4:0]Rs2_addr_ID, //译码阶段寄存器读地址 2
input Rs1_used, //Rs1 被使用
input Rs2_used, //Rs2 被使用
```

由于这些内容可以直接从指令中读出，因此不用修改 SCPU unit:

```
assign Rs1_addr_ID = Inst_in_ID[19:15];
assign Rs2_addr_ID = Inst_in_ID[24:20];
assign Rs1_used = (Inst_in_ID[6:0] == 7'b0110011) || // R-type 指令
                  (Inst_in_ID[6:0] == 7'b0000011) || // I-type 指令
                  (Inst_in_ID[6:0] == 7'b1100011); // B-type 指令 (
assign Rs2_used = (Inst_in_ID[6:0] == 7'b0110011) || // R-type 指令
                  (Inst_in_ID[6:0] == 7'b0100011) || // S-type 指令
                  (Inst_in_ID[6:0] == 7'b1100011); // B-type 指令
```

(4) ID_reg_Ex :

此寄存器也需要简单的改动，对 NOP_IDEX 信号做出反应，同时输出 valid 信号指出此时是否寄存器有效：

```
module ID_reg_Ex(
    input clk_IDEX, //寄存器时钟
    input rst_IDEX, //寄存器复位
    input en_IDEX, //寄存器使能

    input NOP_IDEX, //插入 NOP 使能
    input valid_in_IDEX, //有效
    input[31:0] Inst_in_IDEX,

    input[31:0] PC_in_IDEX, //PC 输入
    input[4:0] Rd_addr_IDEX, //写目的地址输入
    input[31:0] Rs1_in_IDEX, //操作数 1 输入
    input[31:0] Rs2_in_IDEX, //操作数 2 输入
    input[31:0] Imm_in_IDEX, //立即数输入
    input ALUSrc_B_in_IDEX, //ALU B 输入选择
    input[3:0] ALU_control_in_IDEX, //ALU 选择控制
    input Branch_in_IDEX, //Beq
    input BranchN_in_IDEX, //Bne
    input MemRW_in_IDEX, //存储器读写
    input[1:0] Jump_in_IDEX, //Jal
    input[1:0] MemtoReg_in_IDEX, //写回选择
    input RegWrite_in_IDEX, //寄存器堆读写
```

```

output reg[31:0] PC_out_IDEX, //PC 输出
output reg[31:0] Inst_out_IDEX, // inst 输出
output reg valid_out_IDEX,

output reg[4:0] Rd_addr_out_IDEX, //目的地址输出
output reg[31:0] Rs1_out_IDEX, //操作数 1 输出
output reg[31:0] Rs2_out_IDEX, //操作数 2 输出
output reg[31:0] Imm_out_IDEX , //立即数输出
output reg ALUSrc_B_out_IDEX, //ALU B 选择
output reg[3:0] ALU_control_out_IDEX, //ALU 控制
output reg Branch_out_IDEX, //Beq
output reg BranchN_out_IDEX, //Bne
output reg MemRW_out_IDEX, //存储器读写
output reg[1:0] Jump_out_IDEX, //Jal
output reg[1:0] MemtoReg_out_IDEX, //写回
output reg RegWrite_out_IDEX //寄存器堆读写
);

```

• 逻辑修改:

```

always @(posedge clk_IDEX or posedge rst_IDEX) begin
    if(rst_IDEX == 1'b1)begin
        PC_out_IDEX <= 32'h0;
        Inst_out_IDEX <= 32'h0;
        Rd_addr_out_IDEX <= 5'h0;
        Rs1_out_IDEX <= 32'h0;
        Rs2_out_IDEX <= 32'h0;
        Imm_out_IDEX <= 32'h0;
        ALUSrc_B_out_IDEX <= 1'h0;
        ALU_control_out_IDEX <= 4'h0;
        Branch_out_IDEX <= 1'h0;
        BranchN_out_IDEX <= 1'h0;
        MemRW_out_IDEX <= 1'h0;
        Jump_out_IDEX <= 2'h0;
        MemtoReg_out_IDEX <= 2'h0;
        RegWrite_out_IDEX <= 1'h0;
        valid_out_IDEX <= 1'b1;
    end
    else if(NOP_IDEX) begin
        // else if(NOP_IDEX || valid_in_IDEX == 1'b0) begin
        PC_out_IDEX <= 32'h0;
        Inst_out_IDEX <= 32'h00000013;
        Rd_addr_out_IDEX <= 5'h0;
        Rs1_out_IDEX <= 32'h0;
        Rs2_out_IDEX <= 32'h0;
        Imm_out_IDEX <= 32'h0;
    end
end

```

```

        ALUSrc_B_out_IDEX <= 1'h0;
        ALU_control_out_IDEX <= 4'h0;
        Branch_out_IDEX <= 1'h0;
        BranchN_out_IDEX <= 1'h0;
        MemRW_out_IDEX <= 1'h0;
        Jump_out_IDEX <= 2'h0;
        MemtoReg_out_IDEX <= 2'h0;
        RegWrite_out_IDEX <= 1'h0;

        valid_out_IDEX <= 1'b0;
    end
    else if(en_IDEX == 1'b1) begin
        PC_out_IDEX <= PC_in_IDEX;
        Inst_out_IDEX <= Inst_in_IDEX;
        Rd_addr_out_IDEX <= Rd_addr_IDEX;
        Rs1_out_IDEX <= Rs1_in_IDEX;
        Rs2_out_IDEX <= Rs2_in_IDEX;
        Imm_out_IDEX <= Imm_in_IDEX;
        ALUSrc_B_out_IDEX <= ALUSrc_B_in_IDEX;
        ALU_control_out_IDEX <= ALU_control_in_IDEX;
        Branch_out_IDEX <= Branch_in_IDEX;
        BranchN_out_IDEX <= BranchN_in_IDEX;
        MemRW_out_IDEX <= MemRW_in_IDEX;
        Jump_out_IDEX <= Jump_in_IDEX;
        MemtoReg_out_IDEX <= MemtoReg_in_IDEX;
        RegWrite_out_IDEX <= RegWrite_in_IDEX;
        valid_out_IDEX <= 1'b1;
    end
end
end

```

(5) Ex_reg_Mem :

此后的两个寄存器不需要对 NOP 进行相应,但是可以根据 valid(可以说 valid 的作用约等于之前 NOP 的作用)来判断目前是否需要进行操作:

```

`timescale 1ns / 1ps

module Ex_reg_Mem(
    input clk_EXMem,
    input rst_EXMem,
    input en_EXMem,

    input [31:0]PC_imm_EXMem,
    input valid_in_EXMem,

```

```

input [31:0]Inst_in_EXMem,

input [31:0]PC_in_EXMem,
input [31:0]PC4_in_EXMem,
input [4:0]Rd_addr_EXMem,
input zero_in_EXMem,
input [31:0]ALU_in_EXMem,
input [31:0]Rs2_in_EXMem,
input Branch_in_EXMem,
input BranchN_in_EXMem,
input MemRW_in_EXMem,
input [1:0]Jump_in_EXMem,
input [1:0]MemtoReg_in_EXMem,
input RegWrite_in_EXMem,

output reg[31:0]PC_imm_out_EXMem,
output reg valid_out_EXMem,
output reg[31:0]Inst_out_EXMem,

output reg[31:0]PC_out_EXMem,
output reg[31:0]PC4_out_EXMem,
output reg[4:0]Rd_addr_out_EXMem,
output reg zero_out_EXMem,
output reg[31:0]ALU_out_EXMem,
output reg[31:0]Rs2_out_EXMem,
output reg Branch_out_EXMem,
output reg BranchN_out_EXMem,
output reg MemRW_out_EXMem,
output reg[1:0]Jump_out_EXMem,
output reg[1:0]MemtoReg_out_EXMem,
output reg RegWrite_out_EXMem
);

always @(posedge clk_EXMem or posedge rst_EXMem) begin
    if(rst_EXMem == 1'b1) begin
        PC_imm_out_EXMem <= 32'h0;
        Inst_out_EXMem <= 32'h0;
        PC_out_EXMem <= 32'h0;
        PC4_out_EXMem <= 32'h0;
        Rd_addr_out_EXMem <= 5'h0;
        zero_out_EXMem <= 1'h0;
        ALU_out_EXMem <= 32'h0;
        Rs2_out_EXMem <= 32'h0;
        Branch_out_EXMem <= 1'h0;
    end
end

```

```

        BranchN_out_EXMem <= 1'h0;
        MemRW_out_EXMem <= 1'h0;
        Jump_out_EXMem <= 2'h0;
        MemtoReg_out_EXMem <= 2'h0;
        RegWrite_out_EXMem <= 1'h0;
        valid_out_EXMem <= 1'b1;
    end
else if(valid_in_EXMem == 1'b0) begin
    PC_imm_out_EXMem <= 32'h0;
    Inst_out_EXMem <= 32'h00000013;
    PC_out_EXMem <= 32'h0;
    PC4_out_EXMem <= 32'h0;
    Rd_addr_out_EXMem <= 5'h0;
    zero_out_EXMem <= 1'h0;
    ALU_out_EXMem <= 32'h0;
    Rs2_out_EXMem <= 32'h0;
    Branch_out_EXMem <= 1'h0;
    BranchN_out_EXMem <= 1'h0;
    MemRW_out_EXMem <= 1'h0;
    Jump_out_EXMem <= 2'h0;
    MemtoReg_out_EXMem <= 2'h0;
    RegWrite_out_EXMem <= 1'h0;
    valid_out_EXMem <= 1'b0;
end
else if(en_EXMem) begin
    PC_imm_out_EXMem <= PC_imm_EXMem;
    Inst_out_EXMem <= Inst_in_EXMem;
    PC_out_EXMem <= PC_in_EXMem;
    PC4_out_EXMem <= PC4_in_EXMem;
    Rd_addr_out_EXMem <= Rd_addr_EXMem;
    zero_out_EXMem <= zero_in_EXMem;
    ALU_out_EXMem <= ALU_in_EXMem;
    Rs2_out_EXMem <= Rs2_in_EXMem;
    Branch_out_EXMem <= Branch_in_EXMem;
    BranchN_out_EXMem <= BranchN_in_EXMem;
    MemRW_out_EXMem <= MemRW_in_EXMem;
    Jump_out_EXMem <= Jump_in_EXMem;
    MemtoReg_out_EXMem <= MemtoReg_in_EXMem;
    RegWrite_out_EXMem <= RegWrite_in_EXMem;
    valid_out_EXMem <= 1'b1;
end
end
endmodule

```


(6) Mem_reg_WB :

```
`timescale 1ns / 1ps
module Mem_reg_WB(
    input clk_MemWB, //寄存器时
    input rst_MemWB, //寄存器复位
    input en_MemWB, //寄存器使能
    input[31:0] PC_in_MemWB,
    input[31:0] Inst_in_MemWB,
    inout valid_in_MemWB,

    input[31:0] PC4_in_MemWB, //PC+4 输入
    input[4:0] Rd_addr_MemWB, //写目的地址输入
    input[31:0] ALU_in_MemWB, //ALU 输入
    input[31:0] DMem_data_MemWB, //存储器数据输入
    input[1:0] MemtoReg_in_MemWB, //写回
    input RegWrite_in_MemWB, //寄存器堆读写

    output reg[31:0] PC_out_MemWB,
    output reg[31:0] Inst_out_MemWB,
    output reg valid_out_MemWB,

    output reg[31:0] PC4_out_MemWB, //PC+4 输出
    output reg[4:0] Rd_addr_out_MemWB, //写目的地址输出
    output reg[31:0] ALU_out_MemWB, //ALU 输出
    output reg[31:0] DMem_data_out_MemWB, //存储器数据输出
    output reg[1:0] MemtoReg_out_MemWB, //写回
    output reg RegWrite_out_MemWB //寄存器堆读写;
);
always @(posedge clk_MemWB or posedge rst_MemWB) begin
    if(rst_MemWB == 1'b1) begin
        PC_out_MemWB <= 32'h0;
        Inst_out_MemWB <= 32'h0;
        PC4_out_MemWB <= 32'h0;
        Rd_addr_out_MemWB <= 5'h0;
        ALU_out_MemWB <= 32'h0;
        DMem_data_out_MemWB <= 32'h0;
        MemtoReg_out_MemWB <= 2'h0;
        RegWrite_out_MemWB <= 1'h0;
        valid_out_MemWB <= 1'b1;
    end
    else if(valid_in_MemWB == 1'b0) begin
        PC_out_MemWB <= 32'h0;
        Inst_out_MemWB <= 32'h00000013;
        PC4_out_MemWB <= 32'h0;
```

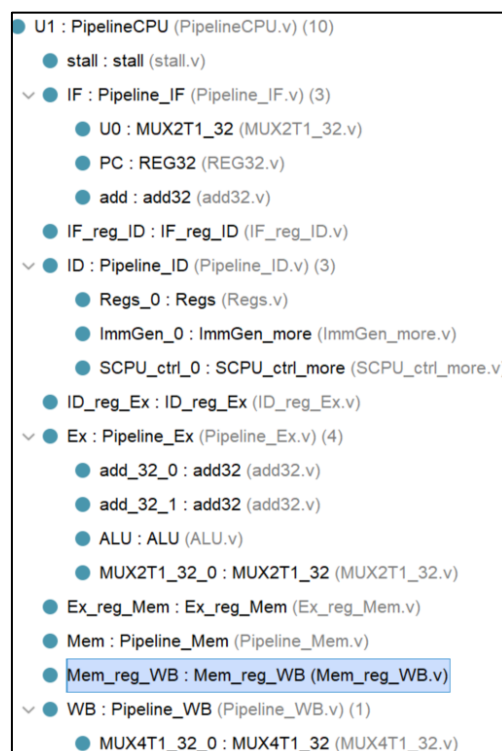
```

        Rd_addr_out_MemWB <= 5'h0;
        ALU_out_MemWB <= 32'h0;
        DMem_data_out_MemWB <= 32'h0;
        MemtoReg_out_MemWB <= 2'h0;
        RegWrite_out_MemWB <= 1'h0;
        valid_out_MemWB <= 1'b0;
    end
    else if(en_MemWB == 1'b1) begin
        PC_out_MemWB <= PC_in_MemWB;
        Inst_out_MemWB <= Inst_in_MemWB;
        PC4_out_MemWB <= PC4_in_MemWB;
        Rd_addr_out_MemWB <= Rd_addr_MemWB;
        ALU_out_MemWB <= ALU_in_MemWB;
        DMem_data_out_MemWB <= DMem_data_MemWB;
        MemtoReg_out_MemWB <= MemtoReg_in_MemWB;
        RegWrite_out_MemWB <= RegWrite_in_MemWB;
        valid_out_MemWB <= 1'b1;
    end
end
endmodule

```

3.2 集成 CPU

通过 3.1 我们通过逻辑代码实现了 Pipeline 对各种冒险的检测以及随后 stall 的处理，首先对各个模块进行集成组合成 PipelineCPU:



【具体代码如下】

```
`timescale 1ns / 1ps

module PipelineCPU(
    input clk,           //时钟
    input rst,           //复位
    input[31:0] Data_in, //存储器数据输入
    input[31:0] inst_IF, //取指阶段指令
    output [31:0] PC_out_IF, //取指阶段 PC 输出
    output [31:0] PC_out_ID, //译码阶段 PC 输出
    output [31:0] inst_ID, //译码阶段指令
    output [31:0] PC_out_IDEX, //执行阶段 PC 输出
    output MemRW_EX, //执行阶段存储器读写
    output MemRW_Mem, //访存阶段存储器读写
    output [31:0] Addr_out, //地址输出
    output [31:0] Data_out, //CPU 数据输出
    output [31:0] Data_out_WB, //写回数据输出
    output [1023:0] all_regs_data //所有寄存器数据输出
);

    wire [31:0] Rs1_out_ID , Rs2_out_ID, Imm_out_ID;
    wire [4:0] Rd_addr_out_ID;
    wire [3:0] ALU_control_ID;
    wire [1:0] MemtoReg_ID, Jump_ID;
    wire ALUSrc_B_ID , Branch_ID , BranchN_ID , MemRW_ID ,
RegWrite_out_ID;

    wire [31:0] PC_out_EX , Rs1_out_IDEX , Rs2_out_IDEX, Imm_out_IDEX;
    wire [4:0] Rd_addr_out_IDEX;
    wire [3:0] ALU_control_IDEX;
    wire [1:0] MemtoReg_IDEX, Jump_IDEX;
    wire ALUSrc_B_IDEX , Branch_IDEX , BranchN_IDEX , MemRW_IDEX ,
RegWrite_IDEX;

    wire [31:0] PC4_out_EX, ALU_out_EX, Rs2_out_EX;
    wire zero_out_EX;

    wire [31:0] PC_out_EXMem , PC4_out_EXMem;
    wire [4:0] Rd_addr_out_EXMem;
    wire [1:0] MemtoReg_out_EXMem, Jump_out_EXMem;
    wire zero_out_EXMem, Branch_out_EXMem, BranchN_out_EXMem,
RegWrite_out_EXMem;

    wire PCSrc; // memory stage
```

```

wire [31:0] PC4_out_MemWB , ALU_out_MemWB, DMem_data_out_MemWB;
wire [4:0] Rd_addr_out_MemWB;
wire [1:0] MemtoReg_out_MemWB;
wire RegWrite_out_MemWB;

wire en_IF , en_IFID , NOP_IFID , NOP_IDEX;

wire[31:0] PC_imm_out_EXMem, Inst_out_EXMem;
wire valid_out_EXMem; // Part 'Ex_reg_Mem'

wire [4:0] Rs1_addr_ID, Rs2_addr_ID;
wire Rs1_used , Rs2_used;

stall stall(
    .rst_stall(rst),
    .Rs1_addr_ID(Rs1_addr_ID),
    .Rs2_addr_ID(Rs2_addr_ID),
    .RegWrite_out_IDEX(RegWrite_IDEX),
    .RegWrite_out_EXMem(RegWrite_out_EXMem),//???
    .Rd_addr_out_IDEX(Rd_addr_out_IDEX),
    .Rd_addr_out_EXMem(Rd_addr_out_EXMem),//???
    .Rs1_used(Rs1_used),
    .Rs2_used(Rs2_used),
    .Branch_ID(Branch_ID),
    .BranchN_ID(BranchN_ID),
    .Jump_ID(Jump_ID),
    .Branch_out_IDEX(Branch_IDEX),
    .BranchN_out_IDEX(BranchN_IDEX),
    .Jump_out_IDEX(Jump_IDEX),
    .Branch_out_EXMem(Branch_out_EXMem),
    .BranchN_out_EXMem(BranchN_out_EXMem),
    .Jump_out_EXMem(Jump_out_EXMem)
    .en_IF(en_IF),
    .en_IFID(en_IFID),
    .NOP_IDEX(NOP_IDEX),
    .NOP_IFID(NOP_IFID)

);

Pipeline_IF IF(
    .clk_IF(clk),
    .rst_IF(rst),
    .en_IF(en_IF),
    .PC_in_IF(PC_imm_out_EXMem),

```

```

        .PCSrc(PCSrc),
        .PC_out_IF(PC_out_IF)
    );

    wire valid_IFID;
    IF_reg_ID IF_reg_ID(
        .clk_IFID(clk),
        .rst_IFID(rst),
        .en_IFID(en_IFID),
        .NOP_IFID(NOP_IFID),
        .PC_in_IFID(PC_out_IF),
        .inst_in_IFID(inst_IF),
        .PC_out_IFID(PC_out_ID),
        .inst_out_IFID(inst_ID),
        .valid_IFID(valid_IFID)
    );

    Pipeline_ID ID(
        .clk_ID(clk),
        .rst_ID(rst),
        .RegWrite_in_ID(RegWrite_out_MemWB),
        .Rd_addr_ID(Rd_addr_out_MemWB),
        .Wt_data_ID(Data_out_WB),
        .Inst_in_ID(inst_ID),
        .Rd_addr_out_ID(Rd_addr_out_ID),
        .Rs1_out_ID(Rs1_out_ID),
        .Rs2_out_ID(Rs2_out_ID),
        .Rs1_addr_ID(Rs1_addr_ID),
        .Rs2_addr_ID(Rs2_addr_ID),
        .Rs1_used(Rs1_used),
        .Rs2_used(Rs2_used),
        .Imm_out_ID(Imm_out_ID),
        .ALUSrc_B_ID(ALUSrc_B_ID),
        .ALU_control_ID(ALU_control_ID),
        .Branch_ID(Branch_ID),
        .BranchN_ID(BranchN_ID),
        .MemRW_ID(MemRW_ID),
        .Jump_ID(Jump_ID),
        .MemtoReg_ID(MemtoReg_ID),
        .RegWrite_out_ID(RegWrite_out_ID),
        .all_regs_data(all_regs_data)
    );

    wire[31:0] Inst_out_IDEX;
    wire valid_out_IDEX;

```

```

ID_reg_Ex ID_reg_Ex(
    .clk_IDEX(clk),
    .rst_IDEX(rst),
    .en_IDEX(1'b1),
    .NOP_IDEX(NOP_IDEX),
    .valid_in_IDEX(valid_IFID),
    .PC_in_IDEX(PC_out_ID),
    .Inst_in_IDEX(inst_ID),
    .Rd_addr_IDEX(Rd_addr_out_ID),
    .Rs1_in_IDEX(Rs1_out_ID),
    .Rs2_in_IDEX(Rs2_out_ID),
    .Imm_in_IDEX(Imm_out_ID),
    .ALUSrc_B_in_IDEX(ALUSrc_B_ID),
    .ALU_control_in_IDEX(ALU_control_ID),
    .Branch_in_IDEX(Branch_ID),
    .BranchN_in_IDEX(BranchN_ID),
    .MemRW_in_IDEX(MemRW_ID),
    .Jump_in_IDEX(Jump_ID),
    .MemtoReg_in_IDEX(MemtoReg_ID),
    .RegWrite_in_IDEX(RegWrite_out_ID),
    .PC_out_IDEX(PC_out_IDEX),
    .Inst_out_IDEX(Inst_out_IDEX),
    .Rd_addr_out_IDEX(Rd_addr_out_IDEX),
    .Rs1_out_IDEX(Rs1_out_IDEX),
    .Rs2_out_IDEX(Rs2_out_IDEX),
    .Imm_out_IDEX(Imm_out_IDEX),
    .ALUSrc_B_out_IDEX(ALUSrc_B_IDEX),
    .ALU_control_out_IDEX(ALU_control_IDEX),
    .Branch_out_IDEX(Branch_IDEX),
    .BranchN_out_IDEX(BranchN_IDEX),
    .MemRW_out_IDEX(MemRW_EX),
    .Jump_out_IDEX(Jump_IDEX),
    .MemtoReg_out_IDEX(MemtoReg_IDEX),
    .RegWrite_out_IDEX(RegWrite_IDEX),
    .valid_out_IDEX(valid_out_IDEX)
);

Pipeline_Ex Ex(
    .PC_in_EX(PC_out_IDEX),
    .Rs1_in_EX(Rs1_out_IDEX),
    .Rs2_in_EX(Rs2_out_IDEX),
    .Imm_in_EX(Imm_out_IDEX),
    .ALUSrc_B_in_EX(ALUSrc_B_IDEX),
    .ALU_control_in_EX(ALU_control_IDEX),

```

```

        .PC_out_EX(PC_out_EX),
        .PC4_out_EX(PC4_out_EX),
        .zero_out_EX(zero_out_EX),
        .ALU_out_EX(ALU_out_EX),
        .Rs2_out_EX(Rs2_out_EX)
    );
    Ex_reg_Mem Ex_reg_Mem(
        .clk_EXMem(clk),
        .rst_EXMem(rst),
        .en_EXMem(1'b1),
        .PC_imm_EXMem(PC_out_EX),
        .PC_in_EXMem(PC_out_IDEX),
        .PC4_in_EXMem(PC4_out_EX),
        .valid_in_EXMem(valid_out_IDEX),
        .Inst_in_EXMem(Inst_out_IDEX),
        .Rd_addr_EXMem(Rd_addr_out_IDEX),
        .zero_in_EXMem(zero_out_EX),
        .ALU_in_EXMem(ALU_out_EX),
        .Rs2_in_EXMem(Rs2_out_EX),
        .Branch_in_EXMem(Branch_IDEX),
        .BranchN_in_EXMem(BranchN_IDEX),
        .MemRW_in_EXMem(MemRW_EX),
        .Jump_in_EXMem(Jump_IDEX), // 'jump' 好一个错误
        .MemtoReg_in_EXMem(MemtoReg_IDEX),
        .RegWrite_in_EXMem(RegWrite_IDEX),
        .PC_imm_out_EXMem(PC_imm_out_EXMem),
        .Inst_out_EXMem(Inst_out_EXMem),
        .valid_out_EXMem(valid_out_EXMem),
        .PC_out_EXMem(PC_out_EXMem),
        .PC4_out_EXMem(PC4_out_EXMem),
        .Rd_addr_out_EXMem(Rd_addr_out_EXMem),
        .zero_out_EXMem(zero_out_EXMem),
        .ALU_out_EXMem(Addr_out),
        .Rs2_out_EXMem(Data_out),
        .Branch_out_EXMem(Branch_out_EXMem),
        .BranchN_out_EXMem(BranchN_out_EXMem),
        .MemRW_out_EXMem(MemRW_Mem),
        .Jump_out_EXMem(Jump_out_EXMem),
        .MemtoReg_out_EXMem(MemtoReg_out_EXMem),
        .RegWrite_out_EXMem(RegWrite_out_EXMem)
    );
    Pipeline_Mem Mem(
        .zero_in_Mem(zero_out_EXMem),
        .Branch_in_Mem(Branch_out_EXMem),

```

```

        .BranchN_in_Mem(BranchN_out_EXMem),
        .Jump_in_Mem(Jump_out_EXMem),
        .PCSrc(PCSrc)
    );

    wire [31:0] PC_out_MemWB , Inst_out_MemWB;
    wire valid_out_MemWB;

    Mem_reg_WB Mem_reg_WB(
        .clk_MemWB(clk),
        .rst_MemWB(rst),
        .en_MemWB(1'b1),
        .PC_in_MemWB(PC_out_EXMem),
        .Inst_in_MemWB(Inst_out_EXMem),
        .valid_in_MemWB(valid_out_EXMem),
        .PC4_in_MemWB(PC4_out_EXMem),
        .Rd_addr_MemWB(Rd_addr_out_EXMem),
        .ALU_in_MemWB(Addr_out),
        .DMem_data_MemWB(Data_in),
        .MemtoReg_in_MemWB(MemtoReg_out_EXMem),
        .RegWrite_in_MemWB(RegWrite_out_EXMem)
        .PC_out_MemWB(PC_out_MemWB),
        .Inst_out_MemWB(Inst_out_MemWB),
        .valid_out_MemWB(valid_out_MemWB),
        .PC4_out_MemWB(PC4_out_MemWB),
        .Rd_addr_out_MemWB(Rd_addr_out_MemWB),
        .ALU_out_MemWB(ALU_out_MemWB),
        .DMem_data_out_MemWB(DMem_data_out_MemWB),
        .MemtoReg_out_MemWB(MemtoReg_out_MemWB),
        .RegWrite_out_MemWB(RegWrite_out_MemWB)
    );

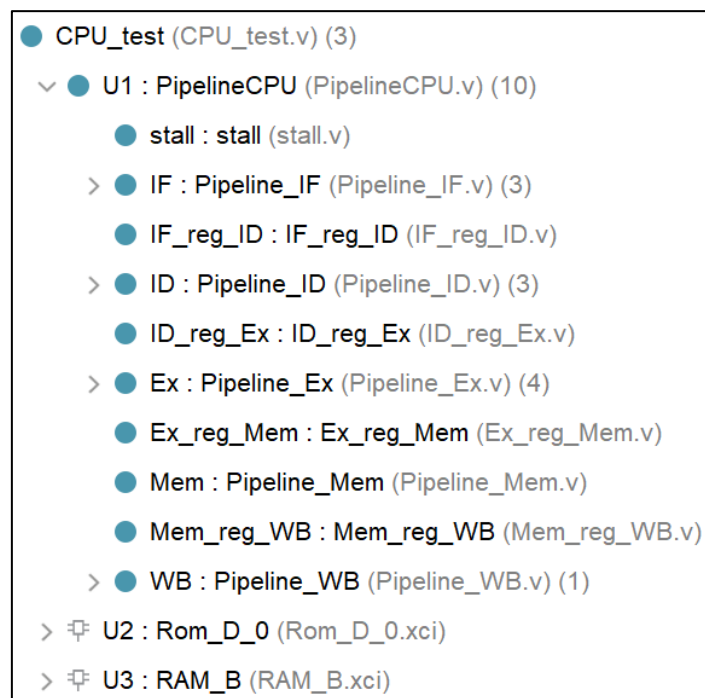
    Pipeline_WB WB(
        .PC4_in_WB(PC4_out_MemWB),
        .ALU_in_WB(ALU_out_MemWB),
        .DMem_data_WB(DMem_data_out_MemWB),
        .MemtoReg_in_WB(MemtoReg_out_MemWB),
        .Data_out_WB(Data_out_WB)
    );
endmodule

```

由于 Pipeline 的对外接口并未修改，因此 CSSTE.v 文件可以沿用 Lab5-3 的代码，不用修改（本报告就不计字数了）。

3.3 搭建仿真平台

参考 Lab5-03 的操作也能建立一个 CPU 仿真测试平台，对相关指令进行检测。利用 PipelineCPU（本实验设计）、RAM、ROM 可以搭建如下图的仿真平台：



初始化 RAM 的数据（D_mem.coe），初始化 ROM 的指令（h_mem.coe），查看 CPU 数据的输入和输出，以及底层的包括寄存器在内的各模块的状态。

因为流水线处理 Stall 的过程在 DEMO 物理测试的时候很难展示在报告中（连续多张图片很难讲清），因此分析的时候就用仿真图来说明，并随后贴出下板时的连续图来与仿真图对应。

（• 仿真图放在后面）。

3.4 DEMO 物理测试

继承替换 CPU 及子模块后，下载.bit 文件，使用 DEMO 程序进行物理验证。本实验由于指令种类多样并且指令较为复杂，通过记录 VGA 中的数据变化，设计了测试记录表格（见实验数据分析）。

- ROM 中填充指令如下：

P_Mem (无冒险)

```

1  memory_initialization_radix=16;
2  memory_initialization_vector=
3  00100093,
4  00100113,
5  00100193,
6  00100213,
7  00802283,
8  00108333,
9  0020C3B3,
10 40110433,
11 05C02483,
12 00327533,
13 00502223,
14 005325B3,
15 0AA3C613,
16 0012D6B3,
17 00147713,
18 0034E7B3,
19 00A50833,
20 0085C8B3,
21 00402903,
22 004629B3,
23 0016DA13,
24 00677AB3,
25 40128B33,
26 00150B93,
27 00986C33,
28 00B9CCB3,
29 0FFA7D13,
30 00390DB3,
31 002A5E33,
32 0AF9EE93,
33 001A0F33,
34 00802F83,
35 F81FF06F,
36 00000033,
37 00000033,
38 00000033;

```

h_Mem (有冒险)

```

1  memory_initialization_radix=16;
2  memory_initialization_vector=
3  00100093,
4  00100113,
5  00100193,
6  00100213,
7  00802283,
8  00128333,
9  0020C3B3,
10 40708433,
11 05C02483,
12 00327533,
13 00502223,
14 005325B3,
15 0AA3C613,
16 00818663,
17 00000013,
18 00000033,
19 0012D6B3,
20 00147713,
21 0034E7B3,
22 00A50833,
23 0085C8B3,
24 00402903,
25 004629B3,
26 0016DA13,
27 00677AB3,
28 01071463,
29 00000013,
30 40128B33,
31 00150B93,
32 00986C33,
33 00B9CCB3,
34 0FFA7D13,
35 00390DB3,
36 002A5E33,
37 0AF9EE93,
38 001A0F33,
39 00802F83,
40 F6DFF06F,
41 00000033,
42 00000033,
43 00000033;

```

- RAM 中储存的数据 (D_mem.coe) 如下：

```

memory_initialization_radix=16;
memory_initialization_vector=
F0000000, 000002AB, 80000000, 0000003F, 00000001, FFF70000,
0000FFFF, 80000000, 00000000, 11111111, 22222222, 33333333,
44444444, 55555555, 66666666, 77777777, 88888888, 99999999,
aaaaaaaa, bbbbbbbb, cccccccc, dddddddd, eeeeeeee, FFFFFFFF,
557EF7E0, D7BDFBD9, D7DBFDB9, DFCFFCFB, DFCFBFFF, F7F3DFFF,
FFFFDF3D, FFFF9DB9, FFFFBCFB, DFCFFCFB, DFCFBFFF, D7DB9FFF,
D7DBFDB9, D7BDFBD9, FFFF07E0, 007E0FFF, 03bdf020, 03def820,
08002300

```

部分测试 VGA 显示图片见实验结果分析

四、实验结果分析

此次物理验证类似 Lab5-3，主要看基本指令能否实现？寄存器数值是否根据对应指令正确修改（存在冒险）？以及流水线是否能正确处理 stall。

PC	Machine	Code Basic	Result
0x0	0x00100093	addi x1 x0 1	x1 = 0x1
0x4	0x00100113	addi x2 x0 1	x2 = 0x1
0x8	0x00100193	addi x3 x0 1	x3 = 0x1
0xc	0x00100213	addi x4 x0 1	x4 = 0x1
0x10	0x00802283	lw x5 8(x0)	x5 = 0x80000000
0x14	0x00128333	add x6 x5 x1	x6 = 0x80000001
0x18	0x0020C3B3	xor x7 x1 x2	x7 = 0x0
0x1c	0x40708433	sub x8 x1 x7	x8 = 0x1
0x20	0x05C02483	lw x9 92(x0)	x9 = 0xFFFFFFFF
0x24	0x00327533	and x10 x4 x3	x10 = 0x1
0x28	0x00502223	sw x5 4(x0)	mem[1] = 0x80000000
0x2c	0x005325B3	slt x11 x6 x5	x11 = 0x0
0x30	0x0AA3C613	xori x12 x7 170	x12 = 0xAA
0x34	0x00818663	beq x3 x8 12	go to 0x40
0x38	0x00000013	addi x0 x0 0	
0x3c	0x00000033	add x0 x0 x0	
0x40	0x0012D6B3	srl x13 x5 x1	x13 = 0x40000000
0x44	0x00147713	andi x14 x8 1	x14 = 0x1
0x48	0x0034E7B3	or x15 x9 x3	x15 = 0xFFFFFFFF
0x4c	0x00A50833	add x16 x10 x10	x16 = 0x2
0x50	0x0085C8B3	xor x17 x11 x8	x17 = 0x1
0x54	0x00402903	lw x18 4(x0)	x18 = 0x80000000
0x58	0x004629B3	slt x19 x12 x4	x19 = 0x0
0x5c	0x0016DA13	srl x20 x13 1	x20 = 0x20000000
0x60	0x00677AB3	and x21 x14 x6	x21 = 0x1
0x64	0x01071463	bne x14 x16 8	go to 0x6c
0x68	0x00000013	addi x0 x0 0	
0x6c	0x40128B33	sub x22 x5 x1	x22 = 0x7FFFFFFF
0x70	0x00150B93	addi x23 x10 1	x23 = 0x2
0x74	0x00986C33	or x24 x16 x9	x24 = 0xFFFFFFFF
0x78	0x00B9CCB3	xor x25 x19 x11	x25 = 0x0
0x7c	0x0FFA7D13	andi x26 x20 255	x26 = 0x0
0x80	0x00390DB3	add x27 x18 x3	x27 = 0x80000001
0x84	0x002A5E33	srl x28 x20 x2	x28 = 0x10000000
0x88	0x0AF9EE93	ori x29 x19 175	x29 = 0xAF
0x8c	0x001A0F33	add x30 x20 x1	x30 = 0x20000001
0x90	0x00802F83	lw x31 8(x0)	x31 = 0x80000000
0x94	0xF6DFF06F	jal x0 -148	go to 0x0
0x98	0x00000033	add x0 x0 x0	
0x9c	0x00000033	add x0 x0 x0	NOP -- 统一Pipeline各阶段
0xa0	0x00000033	add x0 x0 x0	

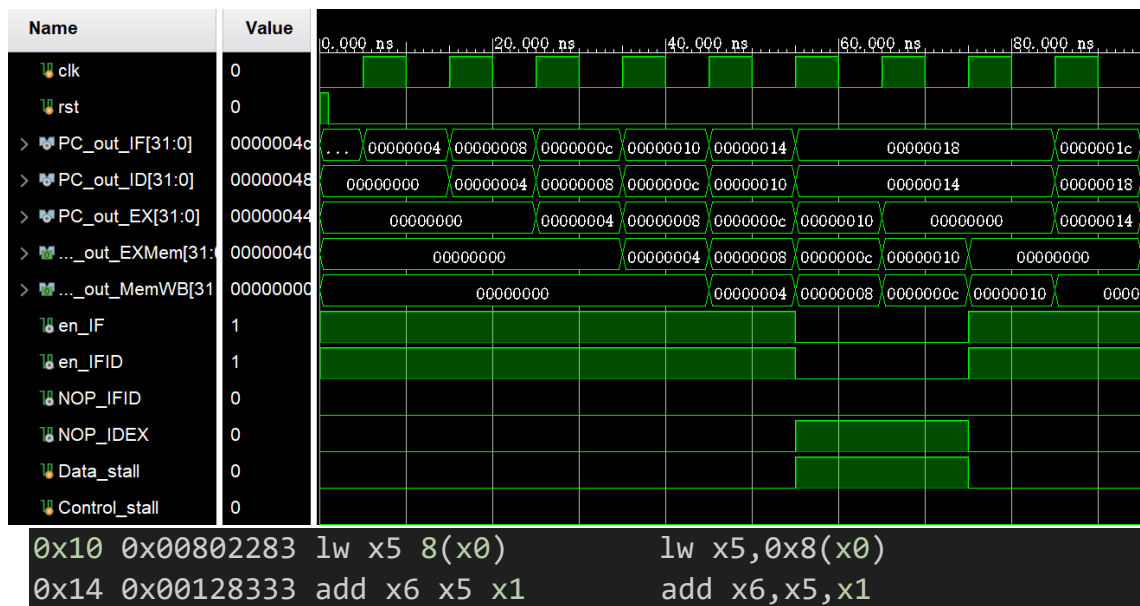
以上表格记录了 h_mem.coe 中的正常程序流（有冒险版本），并记录了运行后各寄存器的数值。

红色的表示此指令与前一指令发生 Data 冒险；

紫色的表示此指令发生 Control 冒险；

黄色的表示 没啥用。

(1) Data Hazard I :



• 可以看到当 PC_out_ID 为 0x14 时（译码得到 Rs1used、Rs2used 等 stall 需要的信号）与 PC_out_EX=0x10 发生 Data Hazard ，此时是 Ex_Hazard。

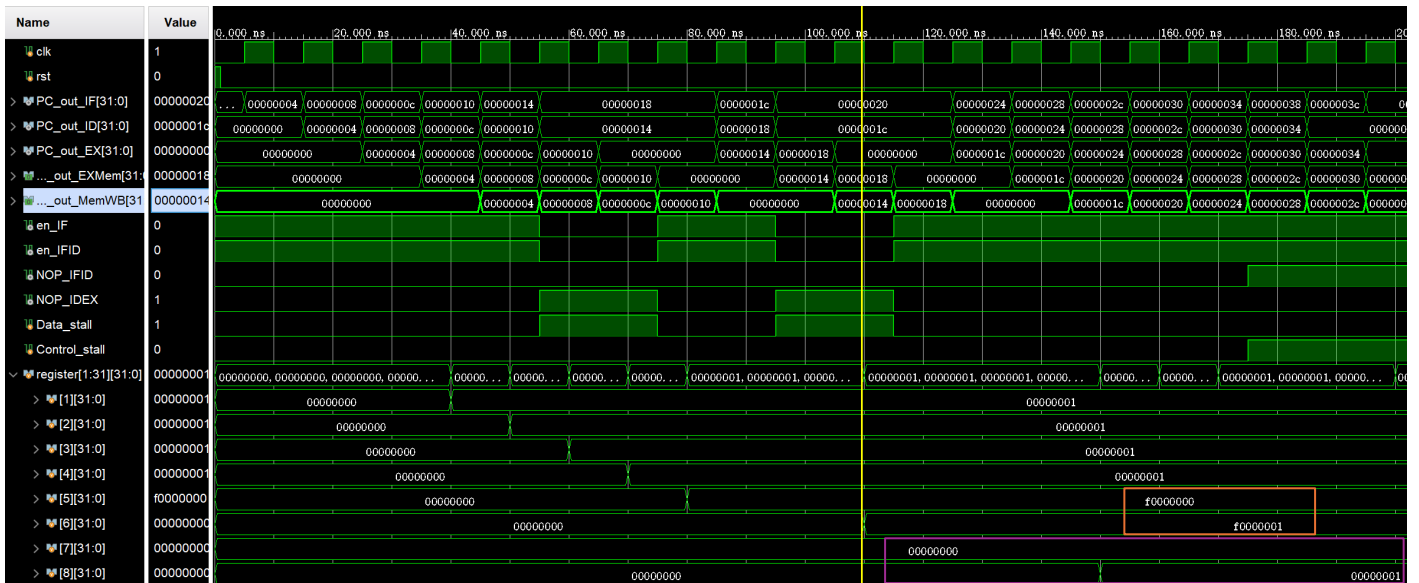
因此插入第 1 次 NOP，IF、ID 阶段停止读入指令/向下传输信号，注意下一 cycle 执行（Ex）阶段变成 NOP；

• 当 PC_out_EXMem = PC_out_Mem = 0x10 时，发生 Mem_Hazard，因此插入第 2 次 NOP，IF、ID 阶段停止读入指令/向下传输信号，下一 cycle 访存（Mem）阶段变成 NOP（此时 lw 数据已经成功读出）；

• 最后写回（WB）阶段 = 0x10，将 x5 的数值成功写回（**注意：ID 阶段对 Regs 的时钟信号进行取反，从而实现一个时钟周期前写后读的操作**），即 75ns 时 WB 写入 x5，80ns 时 ID 读出更新后的 x5，从而令下一 cycle 中 PC_out_EX = 0x14 时使用的 x5 是正确的。Now , Data Hazard I has been solved.

```

Regs Regs_0(
    .clk(~clk_ID), // SIGNIFICANT!
    .rst(rst_ID),
    .Rs1_addr(Inst_in_ID[19:15]),
    .Rs2_addr(Inst_in_ID[24:20]),
    .Wt_addr(Rd_addr_ID),
    .Wt_data(Wt_data_ID),
    .RegWrite(RegWrite_in_ID),
    .Rs1_data(Rs1_out_ID),
    .Rs2_data(Rs2_out_ID),
    .all_regs_data(all_regs_data)
);
  
```



从后续的流水线可以看到（110ns），PC_out_MemWB=0x14 时写入了正确的 x6 数值 0xf0000001（不知道为啥仿真时 x5 load 出来是 0xf0000000，下板则是正确的 0x80000000），而非错误的 0x00000001（未处理冒险的结果）。

以下为 VGA 显示图片（👉👉顺序），后三个阶段只导出了 PC 信号，能够做到简单识别：




```

RV32I Pipelined CPU
===== If =====
pc: 0000001c inst: 40708433 valid: 0
===== Id =====
pc: 00000018 inst: 0020c3b3 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 80000000 t1: 00000000 t2: 00000000 s0: 00000000 s1: 00000000
a0: 00000000 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000014 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen:
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auiopc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000000 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

```

RV32I Pipelined CPU
===== If =====
pc: 00000020 inst: 05c02483 valid: 0
===== Id =====
pc: 0000001c inst: 40708433 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 80000000 t1: 00000000 t2: 00000000 s0: 00000000 s1: 00000000
a0: 00000000 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000018 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen:
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auiopc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000014 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000001 alu_res: 80000001
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

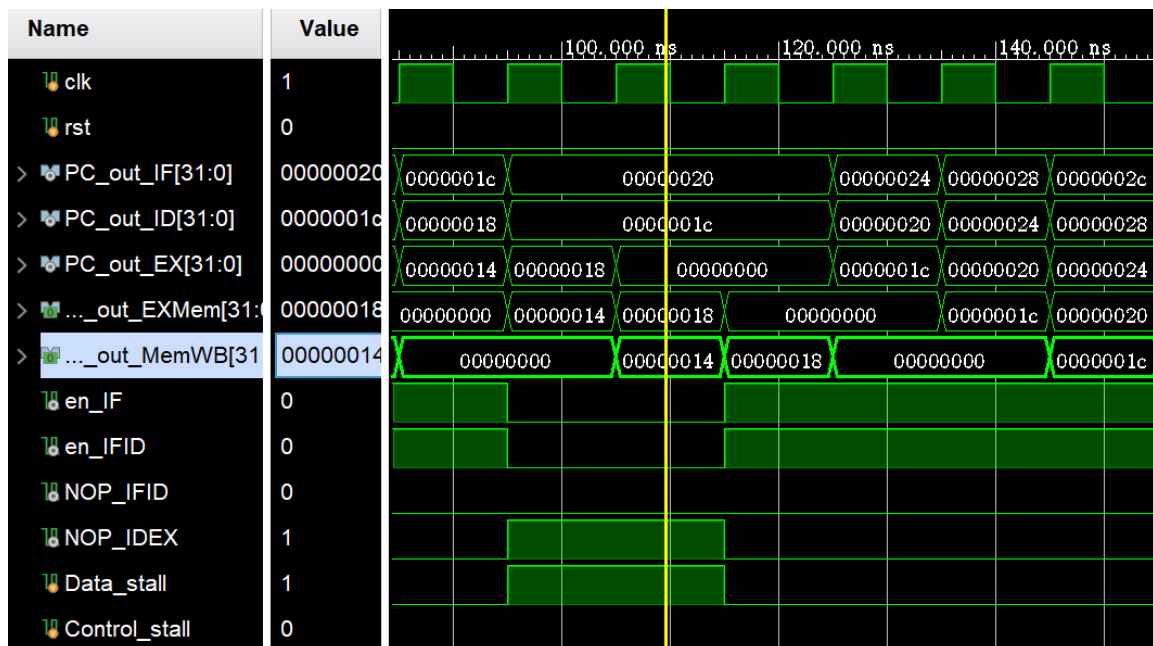
```

RV32I Pipelined CPU
===== If =====
pc: 00000020 inst: 05c02483 valid: 0
===== Id =====
pc: 0000001c inst: 40708433 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 80000000 t1: 80000001 t2: 00000000 s0: 00000000 s1: 00000000
a0: 00000000 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen:
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auiopc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000018 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000001 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000014 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 80000001

```

以上 7 张图展示了处理 Data Hazard 的前因后果，最后 $x6 = t1 = 0x80000001$ 也是修成正果。

(2) Data Hazard II:



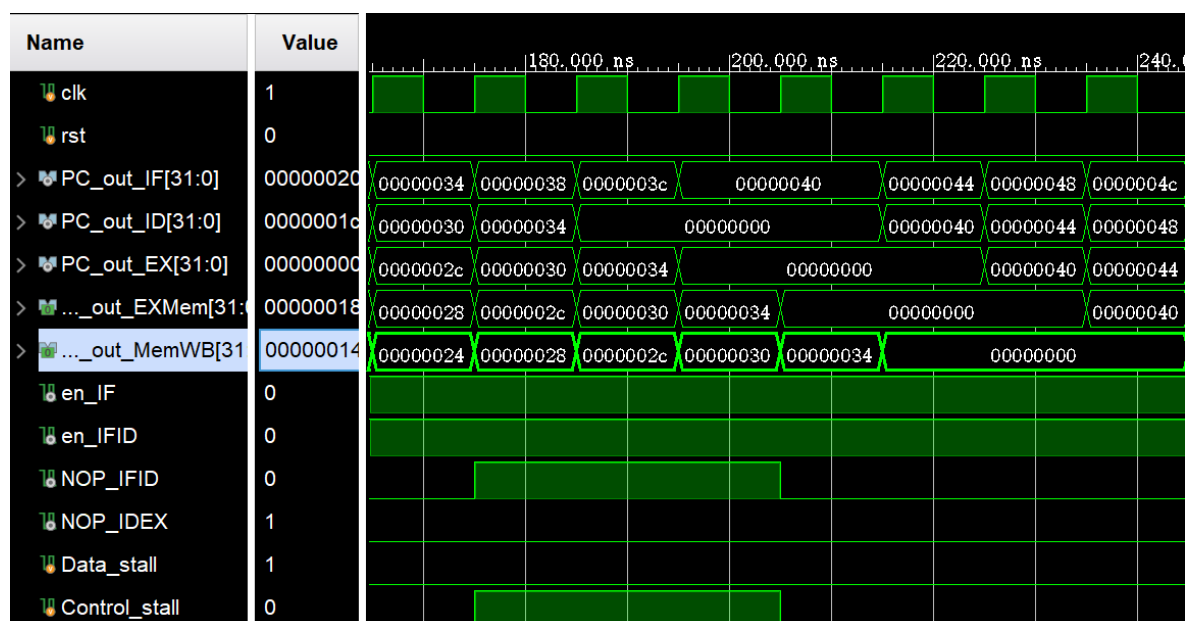
```

0x18 0x0020C3B3 xor x7 x1 x2      xor x7,x1,x2 #x7 = 0
0x1c 0x40708433 sub x8 x1 x7      sub x8,x1,x7 #x8 = 0x1

```

此处对 Data Hazard 的处理与上面一个基本一样，此处就不再详细说明了。
正确数据见 Data Hazard I 第二张图的紫色框框。

(3) Control Hazard I (beq):



```

0x34 0x00818663 beq x3 x8 12      beq x3,x8,loop1
0x38 0x00000013 addi x0 x0 0      addi x0,x0,0x0
0x3c 0x00000033 add x0 x0 x0      add x0,x0,x0
0x40 0x0012D6B3 srl x13 x5 x1     loop1:srl x13,x5,x1

```

首先，处理 Control Hazard 是在 IF 和 ID 之间插入 NOP，即取指（IF）是不会停止的。

可以看到当 PC_out_ID = 0x34 时，译码得到了 Branch_ID = 1（随后三个 cycle 插入 NOP）因此 NOP_IFID = 1，后续阶段不再读取新的指令。

PC_out_EXMem = 0x34 后得到了 PCSrc 来控制下一跳转地址，这可以解释 PC_out_IF 中 0x40 持续两个 cycle 的现象：第一次是正常顺序读取；第二次则是 PCSrc 控制 IF 取指，从而进行 beq 再次跳转到 0x40。

- Control Hazard 解决，可以看到除了 IF 的后续阶段并未执行 0x38、0x3c 两条指令，通过 NOP 直接跳转到 0x40;

```

RV32I Pipelined CPU
===== If =====
pc: 0000003b inst: 00000013
===== Id =====
pc: 00000034 inst: 00819663 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 00000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000030 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auiopc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 0000002c inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000000 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 0000002b inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000004

```

```

RV32I Pipelined CPU
===== If =====
pc: 0000003c inst: 00000033
===== Id =====
pc: 00000000 inst: 00000013 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 00000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000034 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auiopc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000030 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000001 alu_res: 000000aa
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 0000002c inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

```

RV32I Pipelined CPU
===== If =====
pc: 00000040 inst: 0012d6b3
===== Id =====
pc: 00000000 inst: 00000013 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 00000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auiopc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000034 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000001 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000030 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 000000aa

```

```

RV32I Pipelined CPU
===== If =====
pc: 00000040 inst: 0012d6b3
===== Id =====
pc: 00000000 inst: 00000013 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 00000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auiopc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000000 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000034 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

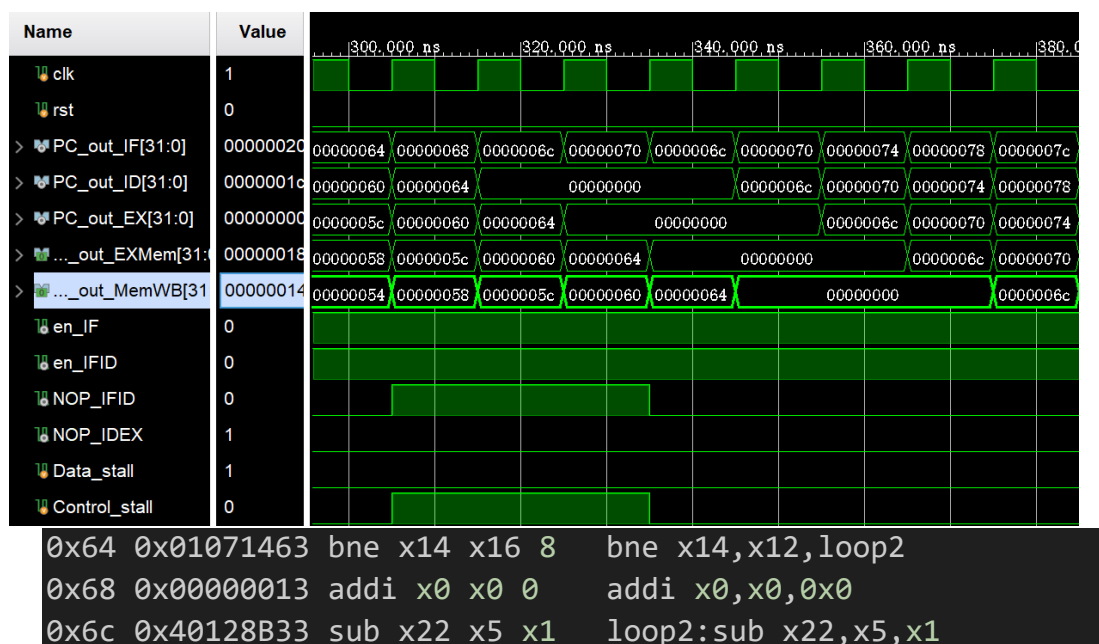
```

RV32I Pipelined CPU
===== If =====
pc: 00000044 inst: 00147713
===== Id =====
pc: 00000040 inst: 0012d6b3 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 00000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auiopc: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000000 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

以上 5 张图片展示了处理 beq 导致的 Control Hazard, 可以看到除了 IF 阶段, 后续 4 个阶段都实现了由 0x34 —— 0x40 的跳转, 没有对中间指令的冗余处理。

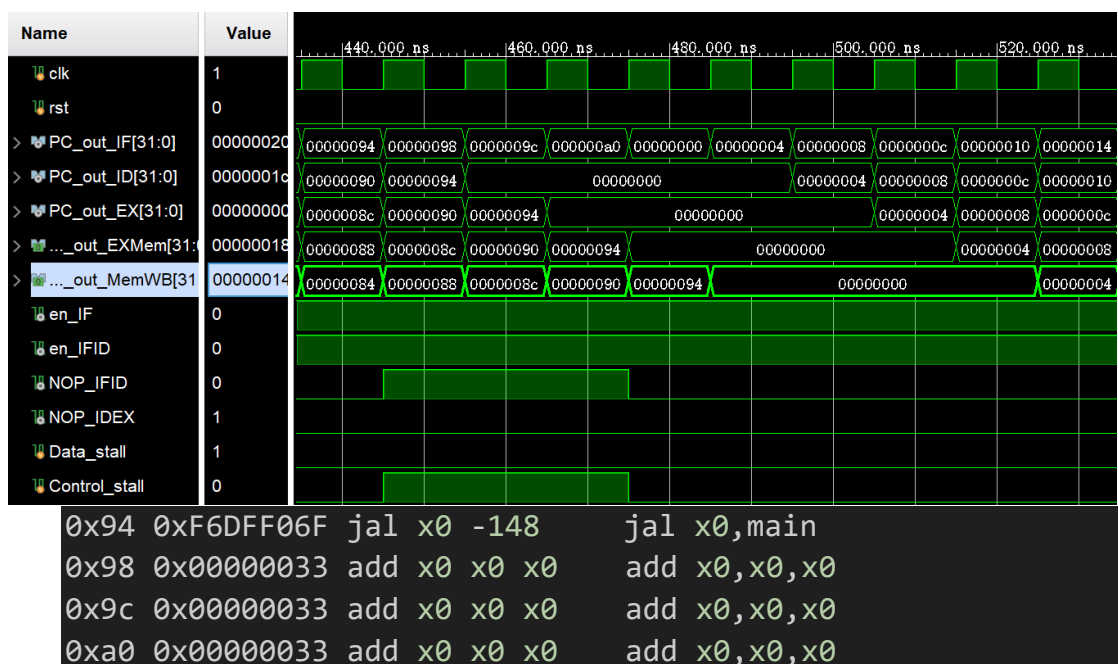
(4) Control Hazard II (bne):



此处处理与 beq 一样，但不同的是此时 bne 跳转指令只跳了 1 条，导致 IF 取到了超前的指令（0x70），怎么办？？？

没事，反正这没传到后续阶段，并且 PCSrc 随后会控制 IF 重新读取正确的下一条指令 0x6c，从此走上光明的道路。

(5) Control Hazard II (jal):



可以看到此处 jal 将后续 3 条指令都跳转了，且后续阶段也未将这 3 个无效指令读取，Control Hazard 也得到了处理。

```

RV32I Pipelined CPU
===== If =====
pc: 00000098 inst: 00000033
===== Id =====
pc: 00000094 inst: f6dfff0f valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 80000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 40000000 a4: 00000001
a5: ffffffff a6: 00000002 a7: 00000001 s2: 00000000 s3: 00000000
s4: 20000000 s5: 00000001 s6: 7fffffff s7: 00000002 s8: ffffffff
s9: 00000000 s10: 00000000 s11: 80000001 t3: 10000000 t4: 000000af
t5: 00000000 t6: 00000000
===== Ex =====
pc: 00000090 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipec: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 0000008c inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000001 alu_res: 20000001
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000088 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 000000af

```

```

RV32I Pipelined CPU
===== If =====
pc: 0000009c inst: 00000033
===== Id =====
pc: 00000000 inst: 00000013 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 80000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 40000000 a4: 00000001
a5: ffffffff a6: 00000002 a7: 00000001 s2: 00000000 s3: 00000000
s4: 20000000 s5: 00000001 s6: 7fffffff s7: 00000002 s8: ffffffff
s9: 00000000 s10: 00000000 s11: 80000001 t3: 10000000 t4: 000000af
t5: 20000001 t6: 00000000
===== Ex =====
pc: 00000094 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipec: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000090 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000001 alu_res: 00000008
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 0000008c inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 20000001

```

```

RV32I Pipelined CPU
===== If =====
pc: 000000a0 inst: 00000033
===== Id =====
pc: 00000000 inst: 00000013 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 80000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 40000000 a4: 00000001
a5: ffffffff a6: 00000002 a7: 00000001 s2: 00000000 s3: 00000000
s4: 20000000 s5: 00000001 s6: 7fffffff s7: 00000002 s8: ffffffff
s9: 00000000 s10: 00000000 s11: 80000001 t3: 10000000 t4: 000000af
t5: 20000001 t6: 00000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipec: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000094 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 40000000 alu_res: ffffffff
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000090 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

```

RV32I Pipelined CPU
===== If =====
pc: 00000000 inst: 00100093
===== Id =====
pc: 00000000 inst: 00000013 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 00000000 t1: 80000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 40000000 a4: 00000001
a5: ffffffff a6: 00000002 a7: 00000001 s2: 00000000 s3: 00000000
s4: 20000000 s5: 00000001 s6: 7fffffff s7: 00000002 s8: ffffffff
s9: 00000000 s10: 00000000 s11: 80000001 t3: 10000000 t4: 000000af
t5: 20000001 t6: 00000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipec: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000000 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000094 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000098

```

```

RV32I Pipelined CPU
===== If =====
pc: 00000004 inst: 00100113
===== Id =====
pc: 00000000 inst: 00100093 valid: 0
x0: 00000000 ra: 00000001 sp: 00000001 gp: 00000001 tp: 00000001
t0: 80000000 t1: 80000001 t2: 00000000 s0: 00000001 s1: ffffffff
a0: 00000001 a1: 00000000 a2: 000000aa a3: 40000000 a4: 00000001
a5: ffffffff a6: 00000002 a7: 00000001 s2: 80000000 s3: 00000000
s4: 20000000 s5: 00000001 s6: 7fffffff s7: 00000002 s8: ffffffff
s9: 00000000 s10: 00000000 s11: 80000001 t3: 10000000 t4: 000000af
t5: 20000001 t6: 80000000
===== Ex =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 rs1: 00 rs2: 00 rs1_val: 00000000 rs2_val: 00000000 reg_wen: 0
is_imm: 0 imm: 00000000 00000000
mem_wen: 0 mem_ren: 0 is_branch: 0 is_jal: 0 is_jalr: 0
is_auipec: 0 is_lui: 0 alu_ctrl: 0 cmp_ctrl: 0
===== Ma =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 mem_w_data: 00000000 alu_res: 00000000
mem_wen: 0 mem_ren: 0 is_jal: 0 is_jalr: 0
===== Wb =====
pc: 00000000 inst: 00000000 valid: 0
rd: 00 reg_wen: 0 reg_w_data: 00000000

```

以上 5 张图片展示了如何处理 jal 引起的 Control Hazard, 可以看到除了 IF 阶段, 后续 4 个阶段都实现了由 0x94 —— 0x00 的跳转。

最后, 这张照片也给出了所有寄存器的数据 (见实验结果分析的第一个表格), 其中 t1、s0 两处涉及 Data Hazard, 均成功处理!

实验成功~~~~

五、讨论与心得

最后一个实验报告也是写完了（不知道谁的 Lab6 报告先交了~~）。

说实话单周期、流水线 CPU 做到后面感觉像是复习？？特别是写报告的时候需要理清 stall 的处理原理还是比较困难的，但最后结合仿真还是搞得比较清楚了吧~~

该复习去了……

~~【并且我宣布 12.20，我自主完成了所有计组实验（真没开挂!）】~~